

VINUNIVERSITY

College of Engineering and Computer Science

COMP1020 – Object-Oriented Programming & Data Structures

Spring 2026

VinTony Coffee POS & Loyalty System

Term Project Final Report

Team VinTony, Group 5

Member 1	Model layer & UML	(customizable ID)
Member 2	Controller layer & History	(customizable ID)
Member 3	Views, Utils & Smoke Tests	(customizable ID)

Submission Deadline: June 2, 2026

Contents

1	Introduction and Project Overview	4
1.1	Problem Statement and Motivation	4
1.2	Project Objectives and Scope	4
1.3	Key Features and Functionalities	4
1.4	Modifications Since the Proposal Stage	5
2	System Requirements and Specifications	6
2.1	Functional Requirements	6
2.2	Non-Functional Requirements	6
3	System Design and Architecture	7
3.1	High-Level Architecture	7
3.2	OOP Concepts Applied	7
3.3	Main Class Interactions	8
4	Data Structures and Algorithms	9
4.1	Selected Data Structures	9
4.2	Algorithms and Their Roles	9
4.3	Trade-offs and Optimization Decisions	9
5	Implementation Details	11
5.1	Language, Libraries, and Build System	11
5.2	File and Package Structure	11
5.3	Key Implementation Details	11
5.3.1	Generic ActionHistory<T>	11
5.3.2	Polymorphic Ingredient Deduction via Decorator Pattern	12
6	Testing and Evaluation	15
6.1	Testing Methodology	15
6.2	Test Cases	15
6.3	Correctness, Robustness, and Usability	15
7	Challenges and Solutions	17
7.1	Technical Difficulties	17
7.2	Design and Architectural Issues	17
7.3	Team Collaboration and Lessons Learned	17
8	Conclusion and Limitations	18
8.1	Overall Achievements	18
8.2	Strengths of the Developed System	18
8.3	Current Limitations	18
8.4	Possible Future Improvements	18
9	Team Contributions	19
10	References	20

11 Appendix	21
11.1 Smoke Test Summary	21
11.2 UML Class Diagram	21
11.3 Order Lifecycle Flowchart	21

1. Introduction and Project Overview

1.1. Problem Statement and Motivation

Small and mid-sized coffee shops in Vietnam typically rely on manual processes for order management, customer tracking, and inventory control. Staff write orders on paper, loyalty cards are stamped manually, and ingredient levels are monitored by eye. This approach is error-prone, slow during peak hours, and makes it nearly impossible to identify high-value (VIP) customers in real time. As café culture grows rapidly and customers expect faster, more personalised service, there is a clear need for an affordable, integrated Point-of-Sale (POS) system tailored to a single-branch operation.

1.2. Project Objectives and Scope

The VinTony Coffee POS & Loyalty System is a desktop application built in Java (JDK 18+) with a Java Swing graphical interface. Its primary goals are to:

- Replace manual order slips with a structured cart-based ordering workflow that supports undo and redo.
- Route orders automatically to a VIP priority queue or a standard FIFO queue based on each customer's loyalty points.
- Award and redeem loyalty points at checkout, and display a ranked leaderboard of top customers.
- Enforce one-time-use discount vouchers with percentage-based discounts.
- Track ingredient inventory in real time, deducting stock when an order is processed and alerting staff when levels fall below defined thresholds.

The system is scoped to a single branch, operates entirely in memory (no external database), and is intended for use by café counter staff rather than end customers.

1.3. Key Features and Functionalities

- **Active Cart Management** – staff add or remove beverages before placing an order; every action is reversible via a generic undo/redo stack capped at 30 entries.
- **Dual-Queue Order Routing** – customers with 100 or more loyalty points are routed to a `PriorityQueue` (max-heap); all others join a `LinkedList`-backed FIFO queue.
- **Customer Registration & Loyalty** – customers are identified by phone number; points accumulate automatically on each completed order and can be redeemed for discounts.
- **Voucher System** – staff generate alphanumeric voucher codes with a configurable percentage discount; each code may only be used once.
- **Inventory Management** – a `Singleton InventoryManager` tracks eight default ingredients; stock is auto-deducted polymorphically when a beverage is processed, and a low-stock alert fires when any ingredient drops below its threshold.
- **Hierarchical Menu Tree** – the full menu is represented as an N-ary tree (Coffee and Tea categories, four to eight items each, three sizes per item) and traversed via DFS to populate the UI.
- **Sorting & Search** – `SortUtil` provides TimSort-based sorts for orders (by time, price, loyalty) and customers (by points, name), as well as binary search for fast order and customer lookup.

1.4. Modifications Since the Proposal Stage

Compared to the initial project proposal, the following enhancements were introduced during development:

- **Generic `ActionHistory<T>`** – the undo/redo manager was redesigned as a type-safe generic class using `Consumer<T>` lambda pairs, making it reusable beyond the order context.
- **`SortUtil` module** – added in Phase 5 to consolidate all sorting and binary-search logic, replacing ad-hoc comparator usage scattered across controller classes.
- **Singleton pattern for `InventoryManager`** – originally the inventory was passed as a constructor argument; it was refactored to a Singleton to eliminate redundant instantiation.
- **`OrderQueue` interface** – extracted as a formal Java interface to allow `RegularOrderQueue` and `VIPOrderQueue` to be swapped polymorphically in `OrderController`.

2. System Requirements and Specifications

2.1. Functional Requirements

The system must support the following core operations:

1. **Order Management:** staff can start a new order for a registered customer, add or remove beverages (with toppings) from the active cart, apply a voucher code, and either place or discard the order.
2. **Order Queue:** placed orders enter the correct queue automatically. Staff can trigger `processNext()` to dequeue and complete the highest-priority order. Pending orders can also be cancelled by ID.
3. **Customer Management:** register new customers (name, phone number, optional email); look up existing customers by phone; view a loyalty-point leaderboard sorted in descending order.
4. **Voucher Management:** create vouchers with a code, discount percentage, and expiry flag; apply a voucher to the current order; mark the voucher as used after redemption.
5. **Inventory Management:** view current stock levels for all ingredients; restock any ingredient by a specified amount; receive low-stock alerts when any ingredient's quantity falls below its defined threshold.

2.2. Non-Functional Requirements

- **Performance:** all queue operations (enqueue/dequeue/peek) complete in $O(1)$ or $O(\log n)$ worst-case time; `HashMap`-based lookups for customers, vouchers, and ingredients are $O(1)$ average.
- **Usability:** the Swing GUI is organised into four tabbed panels (Order, Customer, Inventory, Voucher) with clearly labelled buttons and text fields; no external installation beyond the JDK is required.
- **Memory safety:** the `ActionHistory` stack is capped at 30 entries to prevent unbounded growth during long sessions.
- **Input validation:** blank phone numbers, empty names, negative point values, and duplicate voucher codes are rejected with descriptive error messages before any state change occurs.
- **Portability:** the application is built with a shell script (`build.sh`) that compiles sources to an `out/` directory and runs `Main`; no build tool (Maven, Gradle) is required.
- **Scalability:** the MVC separation and the `OrderQueue` interface ensure that adding new queue strategies or beverage categories requires no changes to the view layer.

3. System Design and Architecture

3.1. High-Level Architecture

The application follows the Model-View-Controller (MVC) architectural pattern, organised into five top-level Java packages as detailed in Table 1.

Table 1: MVC Package Roles in the VinTony POS System

Package	Role
<code>model/</code>	Plain Java Objects: Beverage hierarchy, Customer, Order, Voucher, Ingredient, MenuNode/MenuTree
<code>controller/</code>	Business logic: OrderController, CustomerController, InventoryController, VoucherController, queue implementations
<code>factory/</code>	BeverageFactory – creates Beverage instances from string identifiers
<code>util/</code>	ActionHistory<T> (undo/redo), SortUtil (sorting and binary search)
<code>view/</code>	Java Swing: MainFrame (tabbed container), OrderPanel, CustomerPanel, InventoryPanel, VoucherPanel

The `Main` class instantiates all controllers and passes them to `MainFrame`, which distributes references to the individual panels. Panels communicate exclusively through controller methods – they never access model objects directly, preserving the MVC boundary.

3.2. OOP Concepts Applied

A core requirement of the COMP1020 term project is the direct application of key OOP concepts and GoF design patterns. Table 2 outlines how each was realized.

Table 2: OOP Principles and Design Patterns Applied

OOP Principle / Pattern	Implementation Details
Abstraction	<code>Beverage</code> is an abstract class defining <code>calculatePrice()</code> , <code>getDescription()</code> , and <code>getIngredientRequirements()</code> ; <code>OrderQueue</code> is a Java interface.
Inheritance	<code>Coffee</code> and <code>Tea</code> extend <code>Beverage</code> ; <code>ToppingDecorator</code> extends <code>Beverage</code> ; <code>Topping</code> extends <code>ToppingDecorator</code> .
Polymorphism	<code>calculatePrice()</code> and <code>getIngredientRequirements()</code> are overridden per concrete type; <code>OrderController</code> calls these through the <code>Beverage</code> reference.
Encapsulation	<code>Customer</code> exposes <code>addPoints()</code> / <code>redeemPoints()</code> to guard loyalty points; <code>Order</code> exposes <code>addItem()</code> / <code>removeItem()</code> to maintain its item list.
Factory Pattern	<code>BeverageFactory.create(type, size)</code> instantiates the correct <code>Coffee</code> or <code>Tea</code> subclass from string inputs.
Decorator Pattern	<code>ToppingDecorator</code> wraps any <code>Beverage</code> ; <code>Topping</code> (concrete decorator) adds its price and description by delegating to the wrapped object, allowing unlimited stacking.
Singleton Pattern	<code>InventoryManager.getInstance()</code> guarantees a single inventory instance across all controllers.

3.3. Main Class Interactions

When staff place an order, the interaction flows as follows: the `OrderPanel` collects the customer's phone number, calls `CustomerController.findByPhone()` to retrieve the `Customer` object, then calls `OrderController.startNewOrder(customer)`. As items are added via the GUI, `OrderController.addItemToActive(beverage)` is invoked; each call is wrapped in an `ActionHistory` action so it can be undone. When the staff clicks Place Order, `OrderController.placeActiveOrder()` validates the cart, sets the status to `PENDING`, and routes the Order to either `VIPOrderQueue` or `RegularOrderQueue` based on `Customer.isVip()`. Clicking Process Next calls `OrderController.processNext()` which dequeues the highest-priority order, passes ingredient requirements to `InventoryController.consumeIngredients()`, marks the order `COMPLETED`, and calls `Customer.addPoints()` with the earned points.

4. Data Structures and Algorithms

4.1. Selected Data Structures

The system leverages various collections from the Java Collections Framework (JCF) to achieve optimal performance, as detailed in Table 3.

Table 3: Selected Data Structures and Complexities

Structure	JCF Class	Purpose
LinkedList (Queue)	LinkedList	FIFO order queue for regular customers
PriorityQueue	PriorityQueue	Priority queue ordered by loyalty points; FIFO tiebreak by order
HashMap	HashMap	Customer, voucher, and inventory lookup by code/phone
ArrayDeque (Stack $\times 2$)	ArrayDeque	Undo stack and redo stack for cart actions; capped at 30
N-ary Tree	Custom	Hierarchical menu: root $\rightarrow$ Category $\rightarrow$ Item $\rightarrow$ Size (leaf)
ArrayList	ArrayList	Storage for list sorts and dynamic lookup

4.2. Algorithms and Their Roles

- **Max-heap ordering (VIPOrderQueue):** Java's `PriorityQueue` is configured with a custom `Comparator` that sorts by `Customer.getLoyaltyPoints()` descending, with `Order.getOrderTime()` as a tiebreaker. This ensures that the highest-value customer is always served first while fairness is preserved within the same loyalty tier.
- **Depth-First Search (MenuTree):** The `getAllLeaves()` method recursively visits each `MenuNode`. Internal nodes (category, item) are skipped; leaf nodes carry a `Beverage` reference and are collected into an `ArrayList` for display. This design allows the menu to be extended with new categories or items simply by adding nodes to the tree.
- **Lambda-pair Undo/Redo (ActionHistory):** Each action is stored as a pair of `Consumer<T>` lambdas: `doAction` and `undoAction`. Executing an action calls `doAction.accept(target)` and pushes the record onto the `undoStack`; `undo()` pops the record, calls `undoAction.accept(target)`, and pushes it onto the `redoStack`.
- **TimSort (SortUtil):** `SortUtil.sortByPointsDesc()` and related methods copy the input list into a new `ArrayList` and call `List.sort()` with a lambda comparator. Java's TimSort is stable and runs in $O(n \log n)$ worst-case, which is highly efficient.
- **Binary Search (SortUtil):** Used for fast order lookup by ID and customer lookup by name on a pre-sorted list, yielding $O(\log n)$ query time.

4.3. Trade-offs and Optimization Decisions

- **PriorityQueue vs. LinkedList for VIP:** A max-heap adds $O(\log n)$ overhead per enqueue compared to the $O(1)$ of a plain list. This trade-off is justified because VIP orders are few, so n is always small, yet the guaranteed priority ordering significantly improves VIP customer experience.
- **HashMap vs. TreeMap for lookups:** `HashMap` provides $O(1)$ average access, which is optimal for phone/code/ingredient lookups. `TreeMap` would give $O(\log n)$ but offer sorted key iteration – a feature not required here, so `HashMap` was preferred.

- **ArrayList copy in SortUtil:** Each sort creates a shallow copy of the input list, leaving the authoritative data in the controller unchanged. This prevents accidental mutation of live collections and makes the sort a pure function with no side effects.
- **Undo stack cap at 30:** An unbounded undo history would accumulate `Action` objects for the entire session. The cap ensures constant memory overhead while still covering all realistic user actions in a single order build.

5. Implementation Details

5.1. Language, Libraries, and Build System

The project is implemented entirely in Java SE 18. No third-party libraries are used; all data structures are from the Java Collections Framework (`java.util`). The GUI is built with Java Swing (`javax.swing`), which ships with the JDK. A single shell script, `build.sh`, compiles all source files under `src/` to `out/` using `javac` and then launches the application with `java -cp out Main`. This zero-dependency setup means the project runs on any machine with a compatible JDK.

5.2. File and Package Structure

The system layout separates concerns cleanly:

- `src/Main.java` – Entry point: instantiates all controllers and launches `MainFrame`.
- `src/controller/` – Business logic: `OrderController`, `CustomerController`, `InventoryController`, `VoucherController`, `OrderQueue` interface, `RegularOrderQueue`, `VIPOrderQueue`.
- `src/factory/` – `BeverageFactory` – maps string identifiers to concrete `Beverage` instances.
- `src/model/beverage/` – `Beverage` (abstract), `Coffee`, `Tea`, `Size` (enum).
- `src/model/addon/` – `ToppingDecorator` (abstract), `Topping` (concrete decorator), `Topping.ToppingType` (enum).
- `src/model/customer/` – `Customer`.
- `src/model/inventory/` – `InventoryManager` (Singleton), `Ingredient`.
- `src/model/menu/` – `MenuTree`, `MenuNode`.
- `src/model/order/` – `Order`, `OrderStatus` (enum).
- `src/model/voucher/` – `Voucher`.
- `src/util/` – `ActionHistory<T>`, `SortUtil`.
- `src/view/` – `MainFrame`, `OrderPanel`, `CustomerPanel`, `InventoryPanel`, `VoucherPanel`.
- `test/` – `VinTonyFeatureSmokeTest.java` – headless integration test.
- `docs/` – `VinTony_UML_Class_Diagram.html`, `order_processing.html`.

5.3. Key Implementation Details

5.3.1 Generic `ActionHistory<T>`

The stack-based undo/redo mechanism is implemented as a type-safe generic class, entirely decoupled from the domain model. Listings 1 illustrates the core of this class.

```

1 package util;
2
3 import java.util.ArrayDeque;
4 import java.util.Deque;
5 import java.util.function.Consumer;
6
7 public class ActionHistory<T> {
8
9     public static class Action<T> {
10         private final String    description;
11         private final T         target;
12         private final Consumer<T> doAction;
13         private final Consumer<T> undoAction;
14
15         public Action(String description, T target,
16                        Consumer<T> doAction, Consumer<T> undoAction) {
17             this.description = description;
18             this.target      = target;

```

```

19         this.doAction      = doAction;
20         this.undoAction    = undoAction;
21     }
22
23     public void execute() { doAction.accept(target); }
24     public void undo()    { undoAction.accept(target); }
25     public String getDescription() { return description; }
26 }
27
28 private final Deque<Action<T>> undoStack = new ArrayDeque<>();
29 private final Deque<Action<T>> redoStack = new ArrayDeque<>();
30 private final int maxSize;
31
32 public ActionHistory(int maxSize) {
33     this.maxSize = maxSize;
34 }
35
36 public void execute(String description, T target,
37                     Consumer<T> doAction, Consumer<T> undoAction) {
38     Action<T> action = new Action<>(description, target, doAction,
undoAction);
39     action.execute();
40     undoStack.push(action);
41     redoStack.clear(); // Redo is invalidated when a new action is
performed
42
43     if (undoStack.size() > maxSize) {
44         undoStack.removeLast();
45     }
46 }
47
48 public String undo() {
49     if (undoStack.isEmpty()) return null;
50     Action<T> action = undoStack.pop();
51     action.undo();
52     redoStack.push(action);
53     return "Undone: " + action.getDescription();
54 }
55 }

```

Listing 1: Generic Stack-based Undo/Redo Manager in ActionHistory.java

5.3.2 Polymorphic Ingredient Deduction via Decorator Pattern

The Coffee/Tea hierarchy and Toppings employ a highly flexible Decorator pattern. Listing 2 shows how toppings recursively compute drink costs and polymorphic ingredient requirements.

```

1 package model.addon;
2
3 import model.beverage.Beverage;
4 import java.util.LinkedHashMap;
5 import java.util.Map;
6
7 public class Topping extends ToppingDecorator {
8
9     public enum ToppingType {
10         MILK_FOAM("Milk Foam", 8_000),
11         CARAMEL_SYRUP("Caramel Syrup", 10_000),

```

```

12     WHIPPED_CREAM("Whipped Cream", 12_000),
13     PEARL("Pearl (Boba)", 10_000),
14     COCONUT_JELLY("Coconut Jelly", 9_000),
15     EXTRA_SHOT("Extra Espresso", 15_000);
16
17     private final String label;
18     private final double price;
19
20     ToppingType(String label, double price) {
21         this.label = label;
22         this.price = price;
23     }
24     public String getLabel() { return label; }
25     public double getPrice() { return price; }
26 }
27
28 private final ToppingType toppingType;
29
30 public Topping(Beverage beverage, ToppingType toppingType) {
31     super(beverage);
32     this.toppingType = toppingType;
33 }
34
35 @Override
36 public double calculatePrice() {
37     return beverage.calculatePrice() + toppingType.getPrice();
38 }
39
40 @Override
41 public String getDescription() {
42     return beverage.getDescription() + " + " + toppingType.getLabel
43 ();
44 }
45
46 @Override
47 public Map<String, Integer> getIngredientRequirements() {
48     Map<String, Integer> ingredients = new LinkedHashMap<>(beverage
49 .getIngredientRequirements());
50     String ingredientName = switch (toppingType) {
51         case MILK_FOAM -> "Whole Milk";
52         case CARAMEL_SYRUP -> "Caramel Syrup";
53         case WHIPPED_CREAM -> "Whipped Cream";
54         case PEARL -> "Boba Pearls";
55         case COCONUT_JELLY -> "Coconut Jelly";
56         case EXTRA_SHOT -> "Espresso Beans";
57     };
58     int amount = switch (toppingType) {
59         case MILK_FOAM -> 60;
60         case CARAMEL_SYRUP -> 25;
61         case WHIPPED_CREAM -> 30;
62         case PEARL, COCONUT_JELLY -> 40;
63         case EXTRA_SHOT -> 10;
64     };
65     ingredients.merge(ingredientName, amount, Integer::sum);
66     return ingredients;
67 }
68 }

```

Listing 2: Concrete Decorator Class representing toppings in Topping.java

6. Testing and Evaluation

6.1. Testing Methodology

Testing was performed using a dedicated headless smoke test class, `VinTonyFeatureSmokeTest.java`, located in the `test/` directory. This class exercises all major features programmatically – without opening any Swing window – allowing it to be run in a terminal or CI environment. Each test scenario is isolated: controllers are freshly instantiated and data is set up from scratch, so no test depends on the state left by a previous one.

6.2. Test Cases

The testing suite covers all functional areas, ensuring correct integration of patterns and data structures. Table 4 details these tests.

Table 4: System Smoke Test Cases and Results

Test Case	Steps	Expected Outcome	Status
VIP routing	Register customer with 150 pts → place order → inspect queue	Order appears in <code>VIPOrderQueue</code> ; <code>RegularOrderQueue</code> empty	PASS
Regular routing	Register customer with 50 pts → place order → inspect queue	Order appears in <code>RegularOrderQueue</code> ; <code>VIPOrderQueue</code> empty	PASS
Undo / Redo	Add Latte (M) → undo → verify cart empty → redo → verify cart restored	Cart reverts and restores correctly; <code>ActionHistory</code> sizes match	PASS
Voucher	Create 10% voucher → apply to order → verify discount → attempt reuse	Price reduced by 10%; second application rejected as already used	PASS
Inventory	Process order with known ingredients → query stock levels	Stock decreased by the amounts returned by <code>getIngredientRequirements()</code>	PASS
Low-stock alert	Set ingredient quantity to 5, threshold to 50 → check <code>isLowStock()</code>	Returns true; alert displayed in <code>InventoryPanel</code>	PASS
Points award	Process completed order → check customer loyalty points	Points equal to $\text{floor}(\text{totalPrice} / 1000)$ added to customer account	PASS
Cancellation	Place two orders → cancel first by ID → process next	First order marked CANCELLED; second order processed correctly	PASS
Decorator	Latte (M) + Milk Foam + Caramel Syrup → <code>calculatePrice()</code>	Price = 30,000 + 5,000 (M) + 8,000 + 10,000 = 53,000 VND	PASS
Binary search	Sort orders by ID → binary search for a specific ID	Correct Order returned; null returned for non-existent ID	PASS

6.3. Correctness, Robustness, and Usability

- **Correctness:** all ten smoke test scenarios pass without assertion errors on the final build. The decorator stacking test confirms price composition is cumulative and correct.

- **Robustness:** edge cases are handled explicitly – dequeuing from an empty queue throws `NoSuchElementException` with a clear message; redeeming more points than available returns false without mutating the balance; duplicate voucher codes are rejected at creation time.
- **Usability:** the Swing GUI follows a consistent tab-based layout. Buttons are disabled when actions are unavailable (e.g., Undo button is greyed out when the history is empty). Error messages are shown in labelled text fields rather than modal dialogs, avoiding interruption to the workflow.

7. Challenges and Solutions

7.1. Technical Difficulties

- **Challenge: State sync during rapid GUI events** – Maintaining consistent undo/redo state when the GUI could trigger add and remove actions in rapid succession.
Solution: The `ActionHistory<T>` class uses two separate `ArrayDeque` stacks. Every new action clears the redo stack to prevent branching history – the same semantics used by text editors. The lambda-pair design ensures that the captured `Beverage` reference is always the same object that was originally added or removed, so there is no risk of operating on a stale or cloned item.
- **Challenge: Multi-topping ingredient deduction** – Deducting the correct ingredient amounts for composed drinks (base beverage + multiple toppings) without coupling the controller to specific beverage types.
Solution: `Beverage.getIngredientRequirements()` returns a `Map` and each decorator merges its own contribution using `Map.merge()` with `Integer::sum` as the merge function. The controller calls this single polymorphic method and passes the resulting map to `InventoryController` – no `instanceof` checks anywhere.

7.2. Design and Architectural Issues

- **Challenge: Class explosion in toppings** – Choosing between inheritance and composition for toppings: adding toppings as subclasses of `Coffee` or `Tea` would require a new class for every combination.
Solution: The Decorator pattern was adopted. `ToppingDecorator` wraps any `Beverage` – including already-decorated objects – so toppings can be stacked arbitrarily (e.g., Latte + Milk Foam + Caramel Syrup) without an exponential class explosion. Adding a new topping type requires only a new `ToppingType` enum constant.
- **Challenge: View and Model decoupling** – Keeping the Swing view layer from becoming entangled with domain logic as panels grew in complexity.
Solution: A strict MVC boundary was enforced: controller methods return primitive values or model objects; panels never import model classes directly; all model mutations go through a controller. This made it straightforward to test the domain logic headlessly in the smoke test.

7.3. Team Collaboration and Lessons Learned

Tasks were divided by architectural layer: one member owned the model and factory packages, another owned the controller and util packages, and a third developed the Swing view layer. Regular code reviews caught inconsistencies in naming conventions early. The main lesson learned was to define interfaces (`OrderQueue`) before implementing concrete classes – doing so up front would have avoided a mid-project refactor when `VIPOrderQueue` was introduced.

8. Conclusion and Limitations

8.1. Overall Achievements

The VinTony Coffee POS & Loyalty System successfully delivers all features specified in the initial project proposal. The final implementation demonstrates: seven distinct data structures from the Java Collections Framework; four OOP design patterns (Factory, Decorator, Singleton, and the interface-based Strategy for queues); a fully functional MVC Swing application with five panels; a generic undo/redo manager reusable across any domain type; and a headless smoke-test suite that validates all major code paths without a GUI. All test cases pass on the submitted build.

8.2. Strengths of the Developed System

- **Clean architectural separation:** the MVC pattern and the `OrderQueue` interface ensure that any panel, controller, or queue implementation can be replaced independently.
- **Extensible beverage hierarchy:** adding a new drink category (e.g., Smoothie) requires only a new `Beverage` subclass and a `MenuTree` branch – no changes to controllers or views.
- **Robust undo/redo:** the generic `ActionHistory<T>` is fully decoupled from the order domain and could be reused in future features.
- **Zero external dependencies:** the project compiles and runs with nothing beyond the JDK, making deployment trivial on any compatible system.

8.3. Current Limitations

- **No data persistence:** all customers, orders, vouchers, and inventory changes are lost when the application is closed, as the system stores everything in memory.
- **Single-branch design:** there is no concept of multiple stations or concurrent users; the application assumes a single cashier terminal.
- **No authentication:** any user who opens the application has full access to all panels including inventory management and voucher creation.
- **Hardcoded menu:** items and prices are defined in source code; there is no runtime interface for adding or removing menu items.
- **No receipt printing:** the system records completed orders in memory but provides no mechanism to print or export a receipt for the customer.

8.4. Possible Future Improvements

- Persist data to a local SQLite database or JSON file so that customer loyalty points and inventory levels survive restarts.
- Add a login screen with role-based access (cashier vs. manager) to restrict sensitive operations such as restocking and voucher creation.
- Build a REST API back-end (Spring Boot) and migrate the UI to a web or mobile front-end to support multi-station and multi-branch deployments.
- Introduce a dynamic menu editor panel that allows managers to add, edit, or archive menu items without recompiling the application.
- Generate and print PDF receipts using the Java `iText` or Apache `PDFBox` library.

9. Team Contributions

The responsibilities were divided cleanly across members. Table 5 summarizes each team member's primary responsibilities throughout the project lifecycle.

Table 5: Team Member Contribution Summary

Team Member	Primary Responsibilities
Member 1	Model layer: Beverage hierarchy (Beverage, Coffee, Tea, Size), Decorator pattern (ToppingDecorator, Topping), BeverageFactory, MenuTree and MenuNode. UML class diagram (VinTony_UML_Class_Diagram.html).
Member 2	Controller layer: OrderController (cart management, queue routing, cancel logic), OrderQueue interface, RegularOrderQueue, VIPOrderQueue. ActionHistory<T> undo/redo manager. Order lifecycle flowchart (order_processing.html).
Member 3	Model layer: Customer, Voucher, Ingredient, InventoryManager (Singleton). Controller layer: CustomerController, VoucherController, InventoryController. SortUtil (sorting and binary search). Smoke test (VinTonyFeatureSmokeTest.java).
All Members	Java Swing view layer (MainFrame, OrderPanel, CustomerPanel, InventoryPanel, VoucherPanel). Integration testing, bug fixes, build script (build.sh), and final report.

10. References

- [1] Oracle Corporation. *Java SE 18 API Documentation*. <https://docs.oracle.com/en/java/javase/18/docs/api/>
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [3] Oracle Corporation. *The Java Tutorials – Collections*. <https://docs.oracle.com/javase/tutorial/collections/>
- [4] Oracle Corporation. *Creating a GUI With Swing*. <https://docs.oracle.com/javase/tutorial/uiswing/>
- [5] Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
- [6] VinUniversity COMP1020 Lecture Slides – *Data Structures and OOP Design Patterns*, Spring 2026.

11. Appendix

11.1. Smoke Test Summary

`VinTonyFeatureSmokeTest.java` executes the ten test scenarios described in Section 6.2 and prints a PASS / FAIL result for each. All ten scenarios pass on the submitted codebase. The full console output is included in the project repository under `test/smoke_test_output.txt`.

11.2. UML Class Diagram

A fully detailed UML class diagram showing all classes, their fields, methods, and relationships (inheritance, association, dependency) is provided as `docs/VinTony_UML_Class_Diagram.html` in the project archive. Key relationships illustrated:

- Beverage $\leftarrow$ Coffee, Tea
- Beverage $\leftarrow$ ToppingDecorator $\leftarrow$ Topping
- OrderController $\rightarrow$ OrderQueue $\leftarrow$ RegularOrderQueue, VIPOrderQueue
- InventoryManager (Singleton) $\rightarrow$ Ingredient

11.3. Order Lifecycle Flowchart

`docs/order_processing.html` contains an interactive flowchart tracing an order from cart creation through queue routing, processing, inventory deduction, and loyalty-point award, including the cancellation branch.